Sprint Report 3

Team Plugged In
David Jarman, Marshall Gaucher, Kevin Worner
Project STBMF
November 7th – December 5th, 2012

Initial Sprint Backlog

For this sprint, we split the backlog into three sections, plus a common workload. Each member was assigned to a set of tasks and committed themselves to complete those tasks by the end of the sprint.

Virtual STB

Marshall Gaucher was assigned to work on the virtual Set Top Box, since he has the most experience with the actual STB firmware.

1. Send CommandLink messages
    1.1. Serialize the message
    1.2. Encrypt the message
    1.3. Calculate checksum
    1.4. Add a header
    1.5. Send to the intended receiver
2. Receive CommandLink messages
    2.1. Parse header
    2.2. Recalculate checksum and verify message integrity
    2.3. Decrypt the message
    2.4. Apply basic logic to handle the message
3. Send and receive one message as a proof of concept
    3.1. UDP
    3.2. TCP (due to the scope of multiplayer environment we have not had a need to fully implement TCP messages)

Game Server

1. Integrate a Logger into the project
2. Update the User Interface
3. Send and receive one message as a proof of concept
4. Integrate BinarySerializer library
5. Create a BinarySerializer compliant class for each message
    5.1. STB initiated messages
    5.2. Game server initiated messages
6. Encrypt messages before sending
7. Decrypt messages that have been received

Virtual APMax

1. Research Reactive Extensions
2. Refractor current code to use Reactive Extensions
3. Forward messages from game server to STB
    3.1. Decrypt message
    3.2. Set unique identifier in message
4. Forward messages from STB to game server

4.1. Encrypt message
5. Update current code to use latest Reactive Extension package

Common Items

1. Write unit tests for each component
2. Continue writing documentation
   2.1. Design
   2.2. Requirements
3. Gather user requirements and fill out backlog
4. Begin stress tests on communication layer between vSTB, vAPMAX, and Game Server

Work Carried to Next Sprint

vSTB

1. Stabilize vSTB communication layer
   a. Network testing with timings and packets to see the limitations of communication layer on a LAN vSTB and WLAN vSTB within our sandbox environment.
      i. Simulates stress on the communication layer so we don't have games crashing in multiplayer environment.
      ii. Provides a reference point to whose prototype may or may not be causing the issue occurring on the multiplayer environment
   b. Provides vSTB Tester interface for vAPMAX and Game Server
      i. vAPMAX and Game Server can validate our sandbox environment, without needing to know the finer details of current game architecture
      ii. Simplifies unit testing
      iii. Generates runtime stats of incoming and outgoing messages using graphs, view/models……
2. Begin redesign of current game architecture to work with multiplayer game environment
   a. Remove load of current files on STB (game state, resources, images)
   b. Continue design of memory mapped resource handler to parse in file and images
   c. Figure out feasible concepts of dynamic runtime UI to work with vAPMAX and Game Server communications
3. Begin work on unit testing
   a. QtTestLib to minimize tedious GUI testing
4. Implement Wireshark dissector plugin (research phase)
   a. Provides extensive tool to generate network statistics documentation on the fly using Cascade Pilot software
   b. Reduces development time of writing all network tests and graphs from scratch
   c. The plugin can work for all Innovative System's network communication protocols across tiers. Additionally it provides actual tests and documentation to support the features of our STBMF system. More importantly provide tractability to our

requirements to ensure stability of a proof of concept product that may or may not go into production in the future.

vAPMax

1. Upgrade current Reactive Extension package to latest and propagate changes throughout prototype
2. Use message type to determine what type of device the destination is

Game Server

All of the cards for the game server have been completed for this sprint. In the coming sprints, some of the cards for the server are:

1. Plugin interface
2. Sending to multiple STBs
3. Database
4. UI rework and attachment
5. Conversion to server

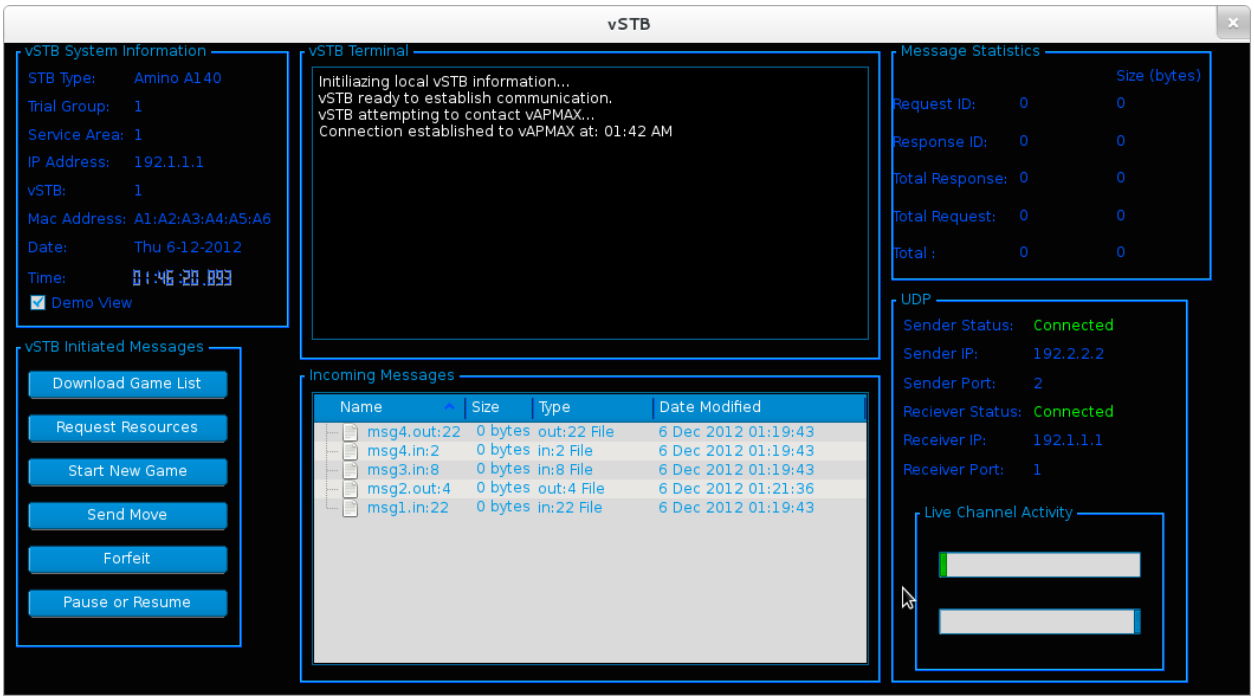
Issues

The only issues from a vSTB standpoint, is the idea of how to deploy the actual game logic to the vSTB from the Game Server. Our design should be able to deploy the logic to the vSTB and the vSTB will only be in charge of sending off the end turn move, while all the intermediate moves are managed by the game logic is stored locally on the vSTB during the lifespan of the game.

Some of the issues we had this sprint for the game server were integrating the serialization library, and getting the blowfish encryption right. The serialization library was a challenge because it was written by an employee who is no longer with the company and there was little documentation about it. The encryption was a challenge because it had at first appeared to work. After a couple days of fiddling with the code on both ends of the communication chain did we realize that we were using the wrong implementation.

Prototypes

vSTB Tester

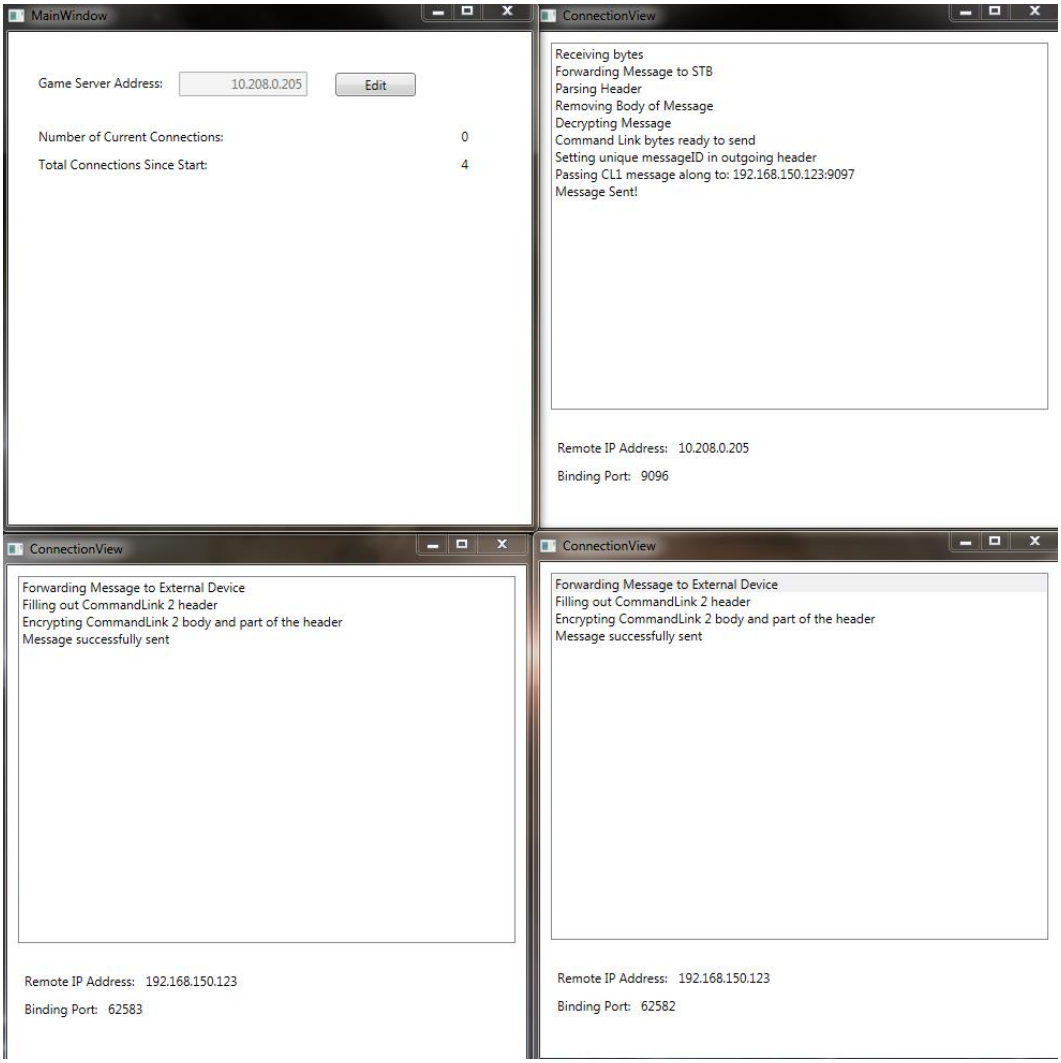


The vSTB Tester will be designed for the vAPMAX and Game Server to simplify communication testing of the sand box environment. This way they can test their new Command Link messages without having to have my vSTB up and running. More importantly it provides graphs and view/models of the incoming and outgoing messages so they no longer have to sit and read console output. The vSTB Tester will provide them with an intuitive tool that will speed up our overall development time across the sandbox environment.

vSTB Multiplayer Environment

The vSTB Multiplayer Environment is still in initial design phase but we do have some feasible ideas that can't be confirmed until we stabilize the communication layer between the vSTB, vAPMAX, and the Game Server. Once we know the limitations of the communication layer we can then refine our design to actually be testable inside the sandbox environment.

vAPMax



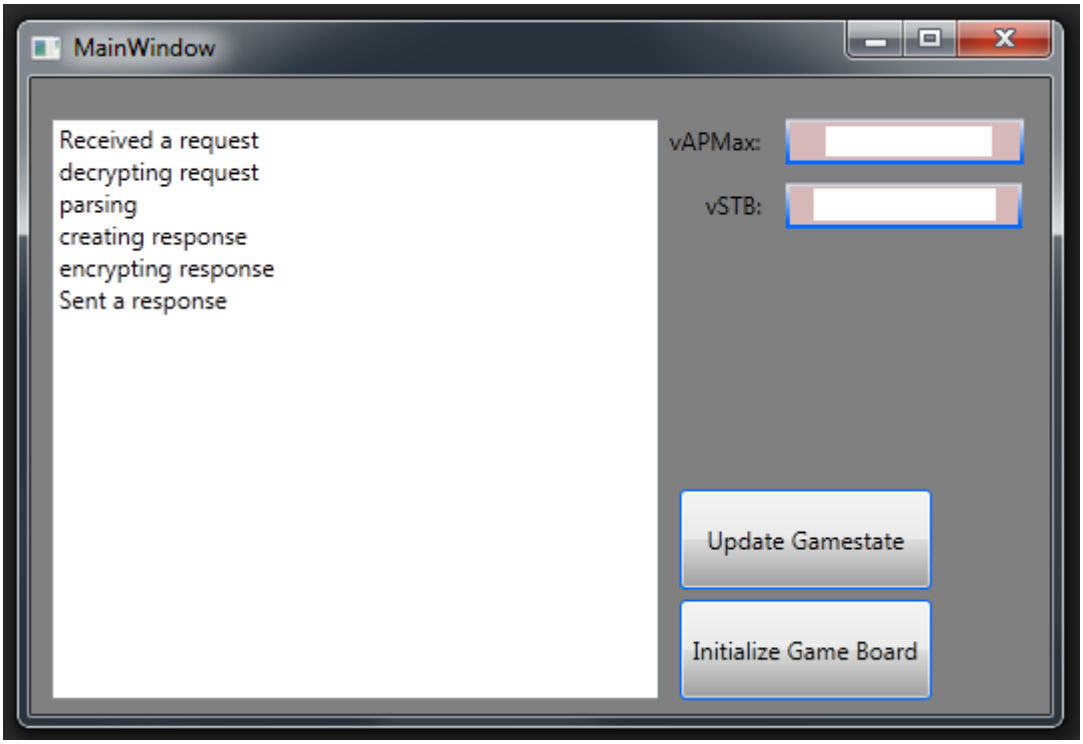
The vAPMax is used to simulate command link message relaying of the actual APMax. During this sprint, David finished up implementing the vAPMax to handle incoming and outgoing UDP and TCP packets.

There are only a couple of small tasks left with the vAPMax which include:

1. Upgrade Rx package to latest
2. Combine the two existing Tcp ports into one port and process based on message type

Game Server

The current game server prototype only supports communication. We felt that before we could move onto the more complicated portions of the project, we had to be confident in the stability of the communication. Because of this, the game server doesn't do a whole lot at the moment. But what it does do, it does well. The UI on the prototype is not how it will look when the project is done, it is constructed as it is for the moment so that it can be more easily used to debug and shore up our communication.


MainWindow
Received a request
decrypting request
parsing
creating response
encrypting response
Sent a response
vAPMax:
vSTB:
Update Gamestate
Initialize Game Board